

Plan de travail — Connecteur custom (métier backend Flask uniquement)

Équipe : **snackeyes** (cœur) + **jj** (modules isolés). Pas de hiérarchie : chacun ses responsabilités, chacun soumet au chef séparément.

Scope strict : métier backend Flask (domaine , sync , storage , config , client SFEC).

Hors scope : app/web (routes, templates, static), Sage BIJOU (F_* , triggers, ODBC).

Référence métier : docs/conception-connecteur-custom/metier-backend.md + modele-metier.puml .

0. Consigne de démarrage (à faire AVANT tout code)

1. **Copier** le connecteur actuel vers le nouveau dossier + **nouveau git** (historique propre).
2. **Nettoyer par stream** (supprimer, pas commenter) :
 - Supprimer app/web/ complet (routes, templates, static) — la vue sera refaite plus tard.
 - Supprimer app/integration/sage/ complet (ODBC, database.py , writer.py Sage).
 - Garder : app/domain/ , app/sync/ (moteur générique), app/storage/ , app/config/ , app/integration/sfec/ , app/core/ , tests/ .
3. **Geler les contrats** (contracts/ , 1 jour ensemble, seule réunion obligatoire) :
schéma des 8 entités, formats FA/AV/TK{:06d} , règles R1-R11, formules, contrat SFEC, interface BillingSystemAdapter , thresholds.json , fixtures FA-PERF-* .
Après signature : **zéro code partagé en cours**, chacun travaille seul.

1. Stream snackeyes — cœur critique (argent, légal, sync)

Lot	Contenu	Fichiers	Gate (vérifiable seul)
A1 Socle	Schéma SQLite + migrations + compteurs + settings	storage/db.py , config/	pytest schema : formats, anti-collision, migrations idempotentes
A2 Moteur factures	create/update/delete , avoirs R1-R3, recalc , whitelist édition	domain/invoices.py	422 sans origine, 422 origine non certifiée, unicité avoir, totaux au centime
A3 Sync + adaptateur	full_sync (certify→contacts→factures→pull), retry/backoff, BillingSystemAdapter + implémentation système propre	sync/ , integration/ (nouveau)	sync complète sur adaptateur mocké, ordre contacts-avant-factures
A4 SFEC	client + payload + validation + polling 12×2s + QR image exacte	integration/sfec/	certif contre mock jouant le contrat réel, QR scannable

2. Stream jj — modules isolés (spec gelée, dicts in / dicts out)

Lot	Contenu	Fichiers	Gate (vérifiable seul)
B1 Lib calcul	calc_line_totals , calc_line_ticket_totals , prorata remise, centime 5%, montant en lettres	domain/pricing.py (nouveau, pur, zéro DB)	100% coverage : zéros, arrondis 0.005 , négatifs rejetés
B2 Validators	NIU (16-17, uppercase auto), email, téléphone, devise, taux	domain/validators.py (nouveau)	table cas OK/KO fournie à l'avance
B3 Seeds + fixtures	60 articles, 14 contacts, 3 vendeurs, taux 18/5/0, INSERT OR IGNORE	domain/seed.py , contracts/fixtures/	seed idempotent (2 runs = même base)
B4 PDF	facture A4 (10 col., 11 totaux, lettres, bloc SFEC) + ticket	domain/pdf.py	PDF générés comparés aux références (texte + QR décodé)
B5 POS ticket	création ticket + contrôle/décrément stock + monnaie + facture miroir (via lib B1)	domain/pos.py	tests sans réseau : rupture bloquée, monnaie exacte, miroir a_comptabiliser

Règle d'or jj : aucun import vers le code de snackeyes (vérifiable :

```
grep -rn "import" domain/pricing.py domain/validators.py ).
```

Si ses tests sont verts, son lot est bon — même si le reste n'existe pas encore.

3. Jalons (soumissions séparées au chef)

Jalon	snackeyes	jj	Intégration
M1	A1 vert	B1+B2 verts (+ tests fournis d'avance)	contrats gelés signés

Jalon	snackeyes	jj	Intégration
M2	A2+A3 sur adaptateur mocké	B3+B4+B5 verts	essai bout-en-bout mocké
M3	A4 SFEC réelle	—	ticket→facture→certif→PDF→QR scannable en réel

4. Garde-fous (sans management)

1. `contracts/` en lecture seule après M0 — tout écart = discussion, pas de surprise.
2. Interdit à jj : DB partagée, sync, réseau, `global`, chemins en dur (tout en config/fixtures).
3. Conventions : `snake_case`, docstring par fonction publique, `py_compile` + `pytest` verts avant toute soumission.
4. En cas de doute métier : le `.md` / `.puml` de conception fait foi, pas les souvenirs.